SE 811: Software Maintenance

Toukir Ahammed

Impact Analysis

Program Dependency Graph

Program Dependency Graph

In the program dependency graph (PDG) of a program:

(i) each simple statement is represented by a node, also called a vertex; and

(ii) each predicate expression is represented by a node.

There are two types of edges:

- data dependency edges
- control dependency edges

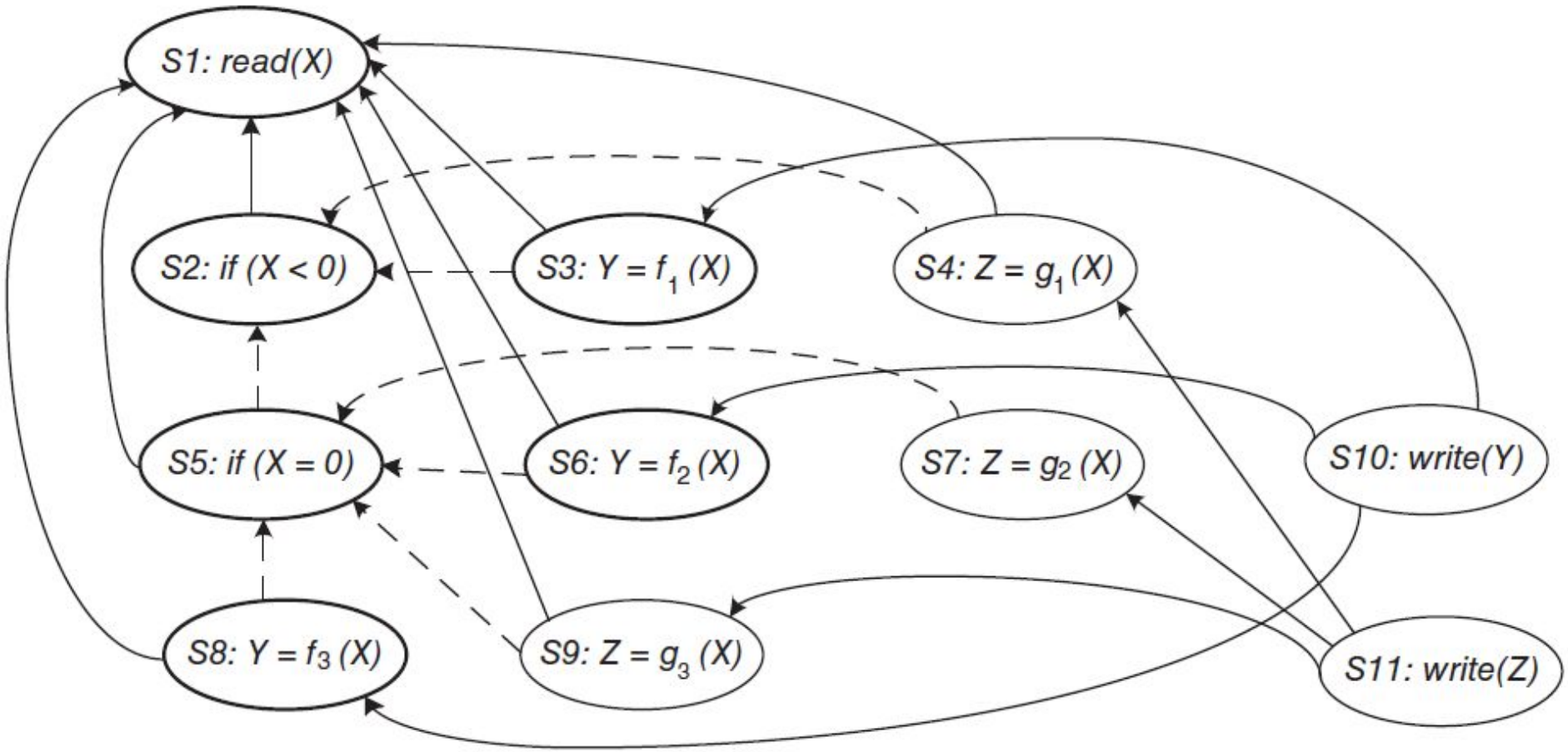
Program Dependency Graph

- Let v_i and v_j be two nodes in a PDG.
- If there is a data dependency edge from node v_i to node v_j, then the computations performed at node v_i are directly dependent upon the results of computations performed at node v_j.
- A control dependency edge from node v_i to node v_j indicates that node v_i may execute based on the result of evaluation of a condition at v_j.

Example Program

```
    begin
S1:     read(X)
S2:     if (X < 0)
        then
S3:         Y = f1 (X);
S4:         Z = g1 (X);
        else
S5:         if (X = 0)
            then
S6:             Y = f2 (X);
S7:             Z = g2 (X);
            else
S8:             Y = f3 (X);
S9:             Z = g3 (X);
            end_if;
        end_if;
S10:    write(Y);
S11:    write(Z);
    end.
```

Program Dependency Graph



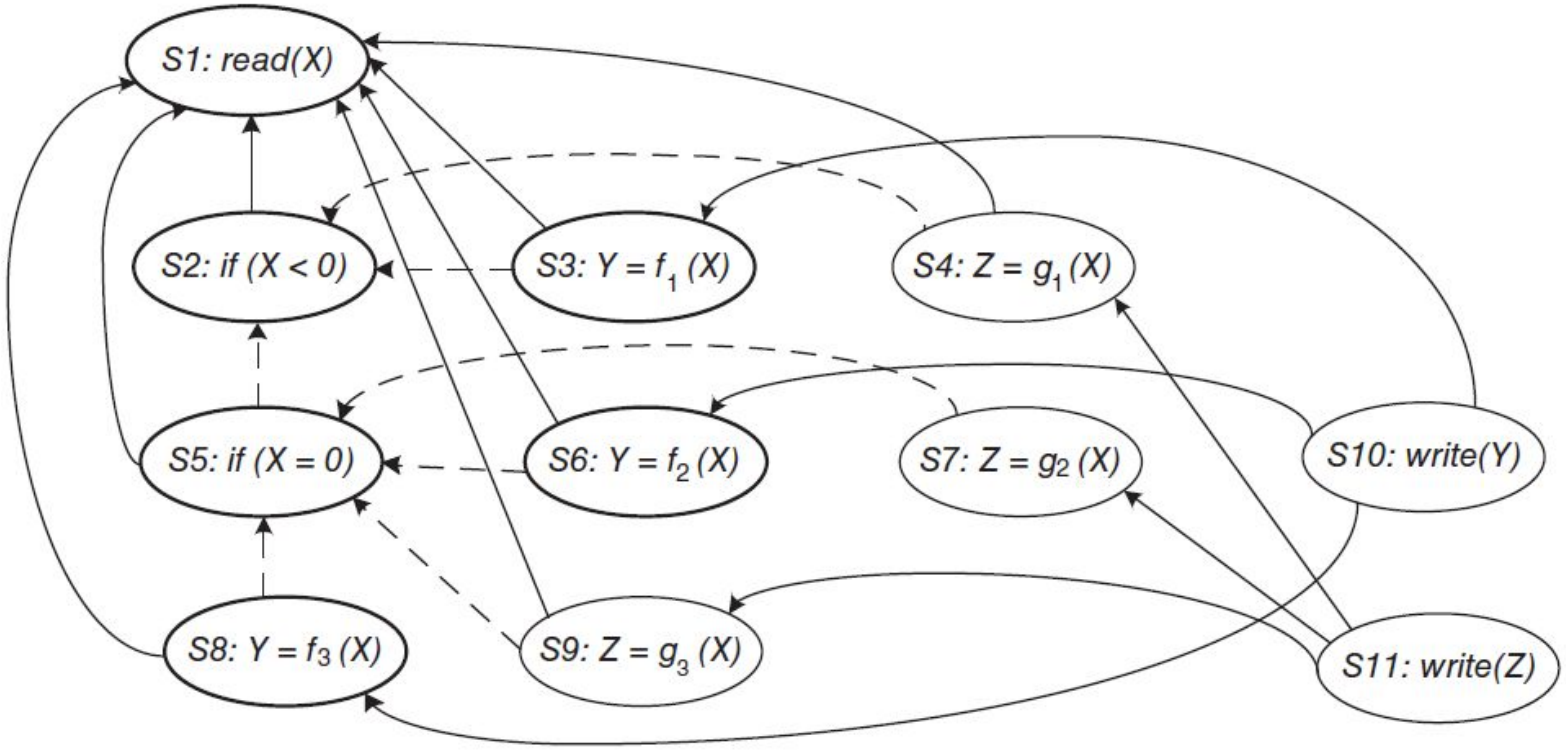

```
    begin
S1:     read(X)
S2:     if (X < 0)
        then
S3:         Y = f1 (X);
S4:         Z = g1 (X);
        else
S5:         if (X = 0)
            then
S6:             Y = f2 (X);
S7:             Z = g2 (X);
            else
S8:             Y = f3 (X);
S9:             Z = g3 (X);
            end_if;
        end_if;
S10:    write(Y);
S11:    write(Z);
    end.
```

Static Program Slice

- A static program slice is identified from a PDG as follows:
  (i) for a variable *var* at node *n*, identify all reaching definitions of *var*; and
  (ii) find all nodes in the PDG which are reachable from those nodes.
- The visited nodes in the traversal process constitute the desired slice.

Static Program Slice

- Consider the program and variable *Y* at *S*10.
- First, find all the reaching definitions of *Y* at node *S*10: *{S*3, *S*6, and *S*8*}*.
- Next, find the set of all nodes which are reachable from *{S*3, *S*6, and *S*8*}*: *{S*1, *S*2, *S*3, *S*5, *S*6, *S*8*}*.



Dynamic Program Slice

- Now, in our example discussed before, the static program slice with respect to variable *Y* at *S*10 for the code contains all the three statements— *S*3, *S*6, and *S*8.
- However, for a given test, one statement from the set *{S*3, *S*6, and *S*8*}* is executed.
- Consider the input value −1 for the variable *X*. For −1 as the value of *X*, only *S*3 is executed.
- A simple way to finding dynamic slices is as follows:
  (i) for the current test, mark the executed nodes in the PDG; and
  (ii) traverse the marked nodes in the graph.

Dynamic Program Slice

- For the case X = −1, the executed nodes are: <S1, S2, S3, S4, S10, S11>.

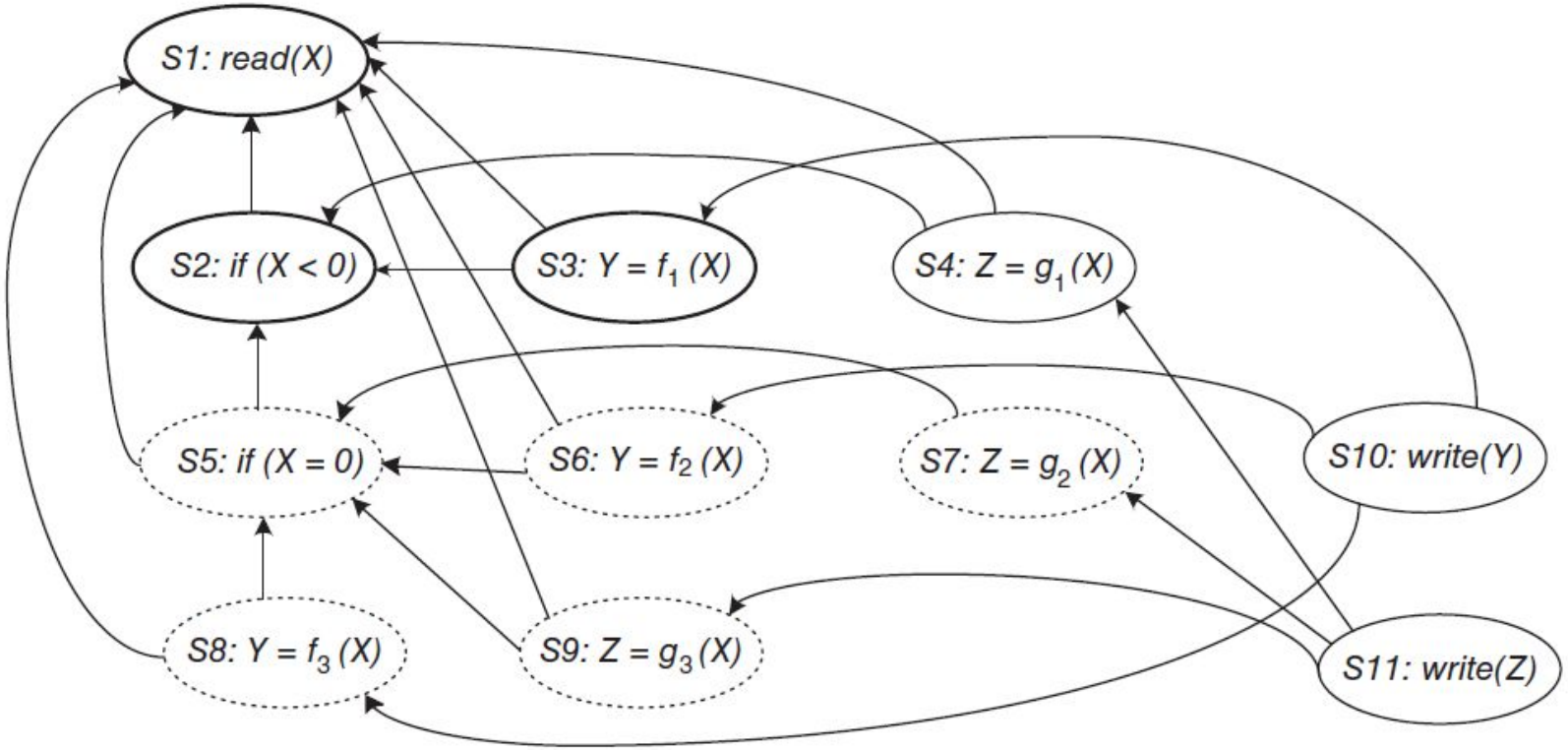


FIGURE 6.11 Dynamic program slice for the code in Figure 6.9, text case $X = -1$, with respect to variable Y

Dynamic Program Slice

- For −1 as the value of *X*, if the value of *Y* is incorrect at *S*10, one can infer that either f_i is erroneous at *S*3 or the "if" condition at *S*2 is incorrect.
- Thus, a dynamic slice is more useful in localizing the defect than the static slice.